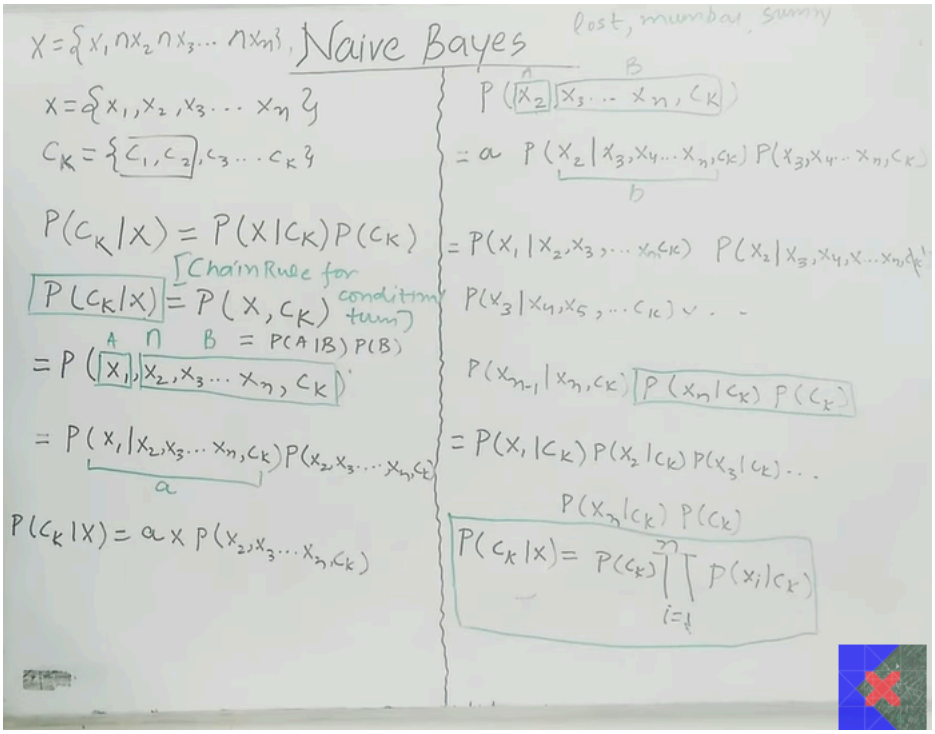



```
# Problem 1
# P(Yes | Outlook=Sunny, Temp=Hot, Humidity=High, Wind=Weak)=?
# P(No | Outlook=Sunny, Temp=Hot, Humidity=High, Wind=Weak)=?
# Prediction: Play = ?
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data=pd.read_csv('/content/sample_data/play_tennis.csv')
data.head()
data.drop(columns=['day'], inplace=True)
data
```

	outlook	temp	humidity	wind	play
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

```
# Solution
#  P(Y/ Sunny, Hot, High, Weak)=P(Sunnny / Y)*P(Hot / Y)*P(High / Y)*P(Weak / Y)*P(Y)
#  P(N/ Sunny, Hot, High, Weak)=P(Sunnny / N)*P(Hot / N)*P(High / N)*P(Weak / N)*P(N)
```

```
#  P(N/ Sunny, Hot, High, Weak)=P(Sunny / N) P(Hot / N) P(High / N) P(Weak / N) P(N)
#  compare and decide using the maximmum a posteriori rule
```

```
# p(Yes)
# P(No)
data['play'].value_counts()
```

play	count
Yes	9
No	5

dtype: int64

```
py=9/14
pn=5/14

print(py)
print(pn)
```

```
0.6428571428571429
0.35714285714285715
```

```
# Outlook
pd.crosstab(data['outlook'], data['play'])
```

play	No	Yes
outlook		
Overcast	0	4
Rain	2	3
Sunny	3	2

```
pon=0
prn=2/5
psn=3/5
poy=4/9
pry=3/9
psy=2/9
```

```
# temp
pd.crosstab(data['temp'], data['play'])
```

play	No	Yes
temp		
Cool	1	3
Hot	2	2
Mild	2	4

```
pcoolNo=1/5
photNo=2/5
pmildNo=2/5
pcoolYes=3/9
photYes=2/9
pmildYes=4/9
```

```
# Humidity

pd.crosstab(data['humidity'], data['play'])
```

play	No	Yes
humidity		
High	4	3
Normal	1	6

```
pHighNo=4/5
pNormalNo=1/5
pHighYes=3/9
pNormalYes=6/9
```

```
# wind
pd.crosstab(data['wind'], data['play'])
```

play	No	Yes
wind		
Strong	3	3
Weak	2	6

```
pStrongNo=3/5
pWeakNo=2/5
pStrongYes=3/9
pWeakYes=6/9
```

```
pyes = py * psy * pHighYes * pWeakYes
pno = pn * psn * photNo * pHighNo * pWeakNo

print(f"P(Yes | Outlook=Sunny, Temp=Hot, Humidity=High, Wind=Weak): {pyes}")
print(f"P(No | Outlook=Sunny, Temp=Hot, Humidity=High, Wind=Weak): {pno}")

if pyes > pno:
    print("Prediction: Play = Yes")
else:
    print("Prediction: Play = No")
```

```
P(Yes | Outlook=Sunny, Temp=Hot, Humidity=High, Wind=Weak): 0.031746031746031744
P(No | Outlook=Sunny, Temp=Hot, Humidity=High, Wind=Weak): 0.02742857142857143
Prediction: Play = Yes
```

```
# MulyNB is aapplicable for categorical data
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

df = pd.DataFrame(data)
# X = df[['outlook','temp','humidity','wind']].copy() # Explicitly create a copy
# y = df['play']
# One-Hot Encoding
X = pd.get_dummies(df[['outlook','temp','humidity','wind']])
y = df['play'].map({'No':0,'Yes':1})


# Encode categorical features
le = LabelEncoder()
for column in X.columns:
    X.loc[:, column] = le.fit_transform(X[column]) # Use .loc for explicit assignment

x_train, x_test, y_train, y_test=train_test_split(X, y, test_size=0.2, random_state=42)

model=MultinomialNB(alpha=0.5)
model.fit(x_train, y_train)
y_pred=model.predict(x_test)
accuracy_score(y_test, y_pred)
# cross_val_score(model, X, y, cv=5)
```

```
# model.predict(X.loc[[0]])
```

```
/tmp/ipython-input-3034972913.py:22: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a futu
  X.loc[:, column] = le.fit_transform(X[column]) # Use .loc for explicit assignment
/tmp/ipython-input-3034972913.py:22: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a futu
  X.loc[:, column] = le.fit_transform(X[column]) # Use .loc for explicit assignment
/tmp/ipython-input-3034972913.py:22: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a futu
  X.loc[:, column] = le.fit_transform(X[column]) # Use .loc for explicit assignment
/tmp/ipython-input-3034972913.py:22: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a futu
  X.loc[:, column] = le.fit_transform(X[column]) # Use .loc for explicit assignment
/tmp/ipython-input-3034972913.py:22: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a futu
  X.loc[:, column] = le.fit_transform(X[column]) # Use .loc for explicit assignment
/tmp/ipython-input-3034972913.py:22: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a futu
  X.loc[:, column] = le.fit_transform(X[column]) # Use .loc for explicit assignment
/tmp/ipython-input-3034972913.py:22: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a futu
  X.loc[:, column] = le.fit_transform(X[column]) # Use .loc for explicit assignment
/tmp/ipython-input-3034972913.py:22: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a futu
  X.loc[:, column] = le.fit_transform(X[column]) # Use .loc for explicit assignment
/tmp/ipython-input-3034972913.py:22: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a futu
  X.loc[:, column] = le.fit_transform(X[column]) # Use .loc for explicit assignment
/tmp/ipython-input-3034972913.py:22: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise in a futu
  X.loc[:, column] = le.fit_transform(X[column]) # Use .loc for explicit assignment
0.6666666666666666
```

GaussianNB

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder
```

2. Loading the Dataset and Preparing Features and Target Variable

```
iris=load_iris()
data=pd.DataFrame(iris.data, columns=iris.feature_names)
data['Special']=iris.target
x=data.drop("Special", axis=1)
y=data['Special']
```

Encoding and Splitting the Dataset

```
le=LabelEncoder()
y=le.fit_transform(y)
x_train, x_test, y_train, y_test=train_test_split(x, y, test_size=0.3, random_state=42)
```

4. Creating and Training the Model

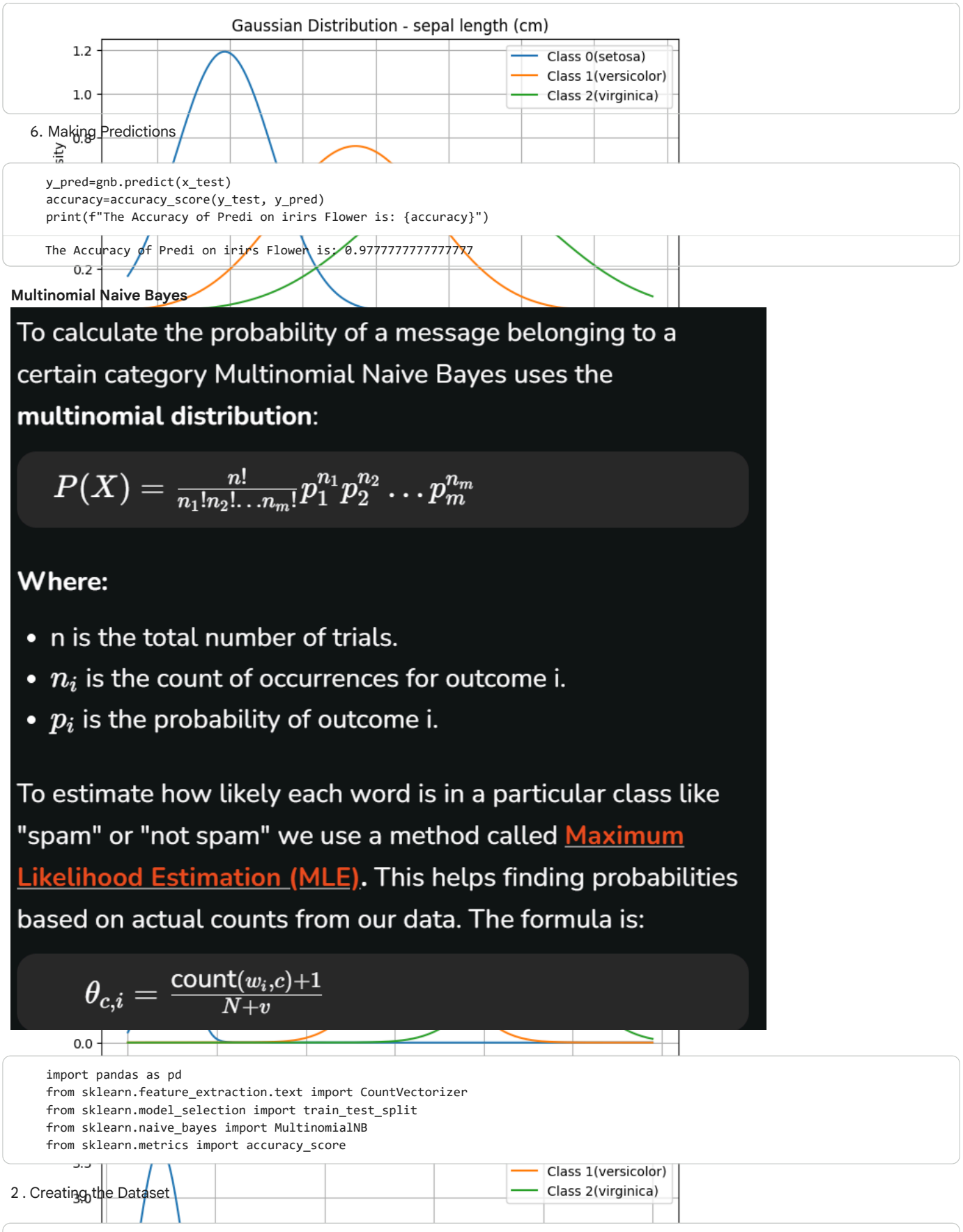
```
gnb=GaussianNB()
gnb.fit(x_train, y_train)
```

```
▾ GaussianNB
GaussianNB()
```

5. Plotting 1D Gaussian Distributions for All Features

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
feature_names=iris.feature_names
num_features=len(feature_names)
num_classes=len(np.unique(y))
x_np=x.to_numpy()
for feature_index in range(num_features):
  feature_name=feature_names[feature_index]
  x_vals=np.linspace(x_np[:, feature_index].min(), x_np[:, feature_index].max(), 200)
  plt.figure(figsize=(8,4))
  for cls in range(num_classes):
```

```
    mean=gnb.theta_[cls, feature_index]
    std=np.sqrt(gnb.var_[cls, feature_index])
    y_vals=norm.pdf(x_vals, mean, std)
    plt.plot(x_vals, y_vals, label=f"Class {cls}({iris.target_names[cls]})")
plt.title(f"Gaussian Distribution - {feature_name}")
plt.xlabel(feature_name)
plt.ylabel("Prob Density")
plt.legend()
plt.grid(True)
plt.show()
```



6. Making Predictions

```
y_pred=gnb.predict(x_test)
accuracy=accuracy_score(y_test, y_pred)
print(f"The Accuracy of Predi on irirs Flower is: {accuracy}")
```

```
The Accuracy of Predi on irirs Flower is: 0.9777777777777777
```

Multinomial Naive Bayes

To calculate the probability of a message belonging to a certain category Multinomial Naive Bayes uses the **multinomial distribution**:

$$P(X) = \frac{n!}{n_1!n_2!\ldots n_m!}p_1^{n_1}p_2^{n_2}\ldots p_m^{n_m}$$

Where:

- n is the total number of trials.
- n_i is the count of occurrences for outcome i.
- p_i is the probability of outcome i.

To estimate how likely each word is in a particular class like "spam" or "not spam" we use a method called Maximum Likelihood Estimation (MLE). This helps finding probabilities based on actual counts from our data. The formula is:

$$\theta_{c,i} = \frac{\text{count}(w_i,c)+1}{N+v}$$

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
```

2 . Creating the Dataset

```
data={
    'text':[
         'Free money now',
        'Call now to claim your prize',
        'Meet me at the park',
        'Let's catch up later',
        'Win a new car today!',
```

```
        'Lunch plans?',
        'Congratulations! You won a lottery',
        'Can you send me the report?',
        'Exclusive offer for you',
        'Are you coming to the meeting?'
    ],
    'label': ['spam', 'spam', 'not spam', 'not spam', 'spam', 'not spam', 'spam', 'not spam', 'spam', 'not spam']
}
df=pd.DataFrame(data)
```

3. Mapping Labels to Numerical Values

```
df['label']=df['label'].map({'spam': 1, 'not spam': 0})
```

4. Splitting the Data

```
x=df['text']
y=df['label']
x_train, x_test, y_train, y_test=train_test_split(x, y, test_size=0.3, random_state=42)
```

5. Vectorizing the Text Data

```
vectorizer=CountVectorizer()
x_train_vectorized=vectorizer.fit_transform(x_train)
x_test_vectorized=vectorizer.transform(x_test)
```

6. Training the Naive Bayes Model

```
model=MultinomialNB()
model.fit(x_train_vectorized, y_train)
```

```
▾ MultinomialNB
MultinomialNB()
```

7. Making Predictions and Evaluating Accuracy

```
y_pred=model.predict(x_test_vectorized)
accuracy=accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100: .2f}%")
```

```
Accuracy:  66.67%
```

8. Predicting for a Custom Message

```
custom_m=["Congratulation, you've won a free vacation"]
print(custom_m)
custom_vec=vectorizer.transform(custom_m)
pred=model.predict(custom_vec)
print("Pred for custom m:","spam" if pred[0]== 1 else "Not Spam")
```

```
["Congratulation, you've won a free vacation"]
Pred for custom m: spam
```

Sentiment Analysis

```
import numpy as np
import pandas as pd
df=pd.read_csv('/content/sample_data/sentiment_analysis.csv')
df.head()
```

	Year	Month	Day	Time of Tweet	text	sentiment	Platform
0	2018	8	18	morning	What a great day!!! Looks like dream.	positive	Twitter
1	2018	8	18	noon	I feel sorry, I miss you here in the sea beach	positive	Facebook
2	2017	8	18	night	Don't angry me	negative	Facebook
3	2022	6	8	morning	We attend in the class just for listening teac...	negative	Facebook
4	2022	6	8	noon	Those who want to go, let them go	negative	Instagram

```
df = df[['text', 'sentiment']]
df
```

	text	sentiment
0	What a great day!!! Looks like dream.	positive
1	I feel sorry, I miss you here in the sea beach	positive
2	Don't angry me	negative
3	We attend in the class just for listening teac...	negative
4	Those who want to go, let them go	negative
...	...	...
494	According to , a quarter of families under six...	negative
495	the plan to not spend money is not going well	negative
496	uploading all my bamboozle pictures of facebook	neutral
497	congratulations ! you guys finish a month ear...	positive
498	actually, I wish I was back in Tahoe. I miss...	negative

499 rows × 2 columns

```
df['text'][0]
```

```
'What a great day!!! Looks like dream.'
```

Text Cleaning

1. Sample 500 rows
2. Remove html tags
3. Remove special characters
4. converting every thing to lower casee
5. Removing stop words
6. Stemming

```
df['sentiment'].replace({'positive': 1, 'negative': 0}, inplace=True)
df.head()
```

	text	sentiment
0	What a great day!!! Looks like dream.	1
1	I feel sorry, I miss you here in the sea beach	1
2	Don't angry me	0
3	We attend in the class just for listening teac...	0
4	Those who want to go, let them go	0

```
import re
clean=re.compile('<.*?>')
df['text'] = df['text'].apply(lambda x: re.sub(clean, '', x))
```

```
def clean_html(text):
  clean=re.compile('<.*?>')
  return re.sub(clean, '', text)
```

```
df['text']=df['text'].apply(clean_html)
```

```
def convert_lower(text):
  return text.lower()
```

```
df['review']=df['review'].apply(convert_lower)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3804         try:
-> 3805             return self._engine.get_loc(casted_key)
   3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'review'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
                                  2 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3810             ):
   3811                 raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
   3813         except TypeError:
   3814             # If we have a listlike key, _check_indexing_error will raise

KeyError: 'review'
```

```
def remove_special(text):
  x=''
  for i in text:
    if i.isalnum():
      x=x+i
    else:
      x=x+' '
  return x
```

```
df['text']=df['text'].apply(remove_special)
```

Bernoulli Naive Bayes

```
import numpy as np
import pandas as pd
from sklearn.naive_bayes import BernoulliNB
from sklearn.feature_extraction.text import CountVectorizer
```

2. Data Analysis

```
import requests
import io
import pandas as pd

# Updated URL for a tab-separated spam/ham dataset from a stable source
url = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv"
response = requests.get(url)
response.raise_for_status()  # Raise an exception for HTTP errors
data = io.StringIO(response.text)

# Read the tab-separated CSV and assign column names
df = pd.read_csv(data, sep='\t', names=['label', 'text'])

# Map 'ham' and 'spam' to numerical values (0 and 1)
df['label_num'] = df['label'].map({'ham': 0, 'spam': 1})
```

```
print(df.shape)
print(df.columns)
# Removed 'df = df.drop(['Unnamed: 0'], axis=1)' as it's not applicable to this dataset
```

```
(5572, 3)
Index(['label', 'text', 'label_num'], dtype='object')
```

3. Count Vectorizer

```
x=df["text"].values
y=df["label_num"].values
cv=CountVectorizer()
x=cv.fit_transform(x)
```

4. Data Splitting, Model Training and Prediction

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.20, random_state=0)
bnb=BernoulliNB(binarize=0.0)
model=bnb.fit(x_train, y_train)
y_pred=bnb.predict(x_test)
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       955
           1       0.99      0.89      0.93       160
```